

avito.tech 

ПОВЕДЕНЧЕСКИЕ ПАТТЕРНЫ

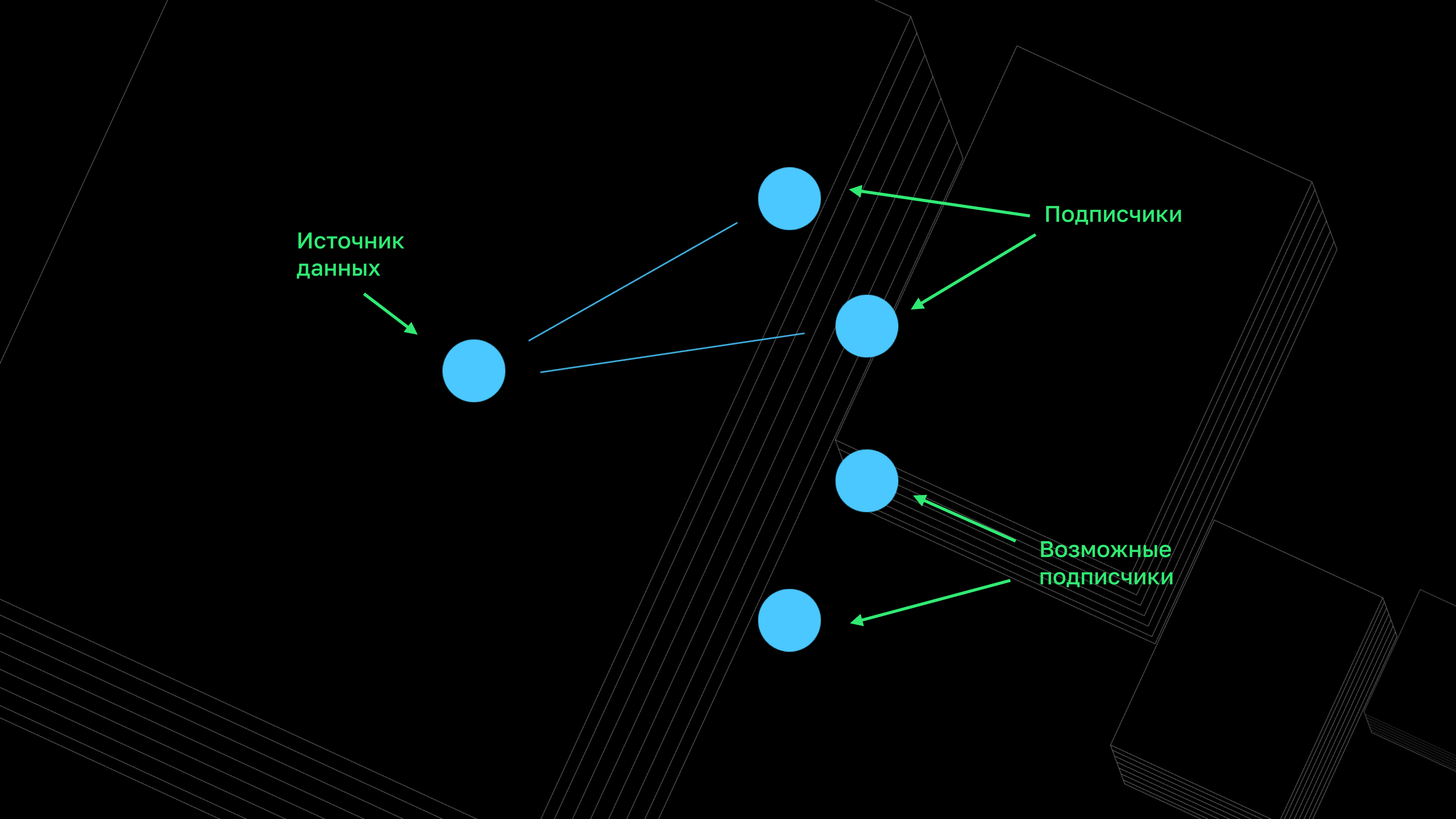
Афанасьев Юра

OBSERVER



НАБЛЮДАТЕЛЬ

Определяет зависимость
типа «один ко многим» между
объектами таким образом, что
при изменении состояния одного
объекта все зависящие
от него оповещаются об этом
и автоматически обновляются



Источник
данных

Подписчики

Возможные
подписчики

Пример из разработки

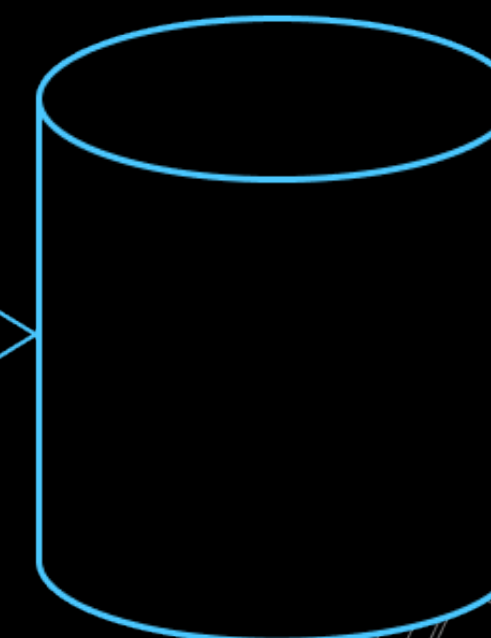
- Инвалидация кеша данных
- Брокер сообщений (AMQP)

getByld

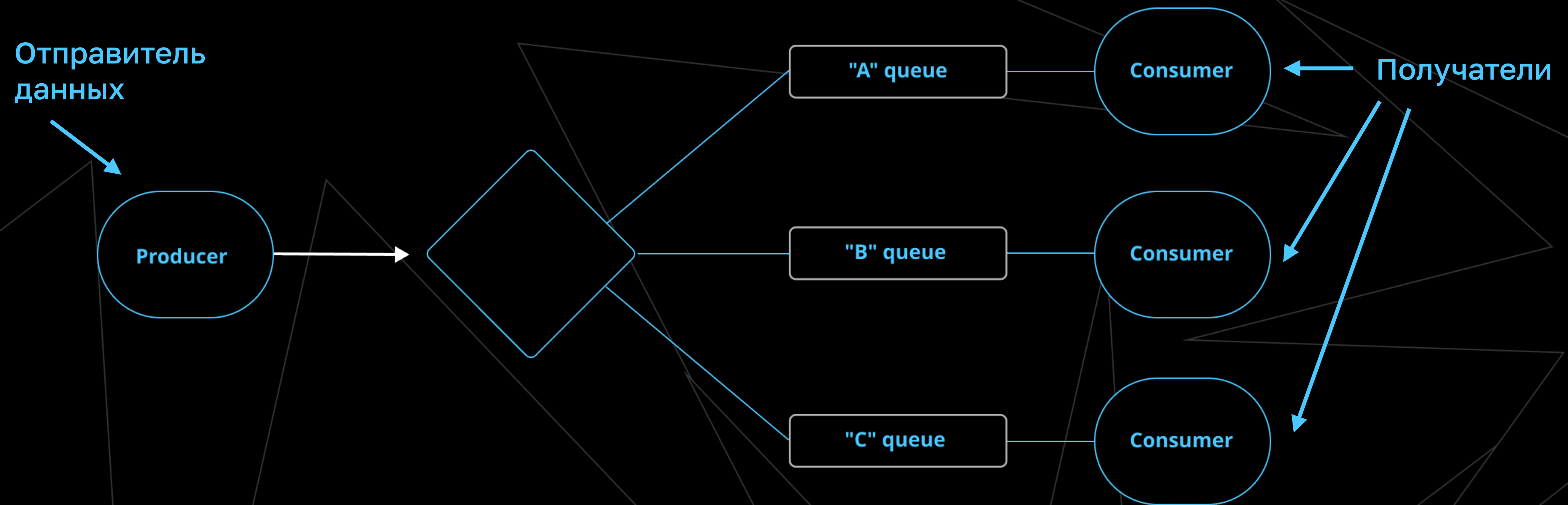
getByUserId

save

cache



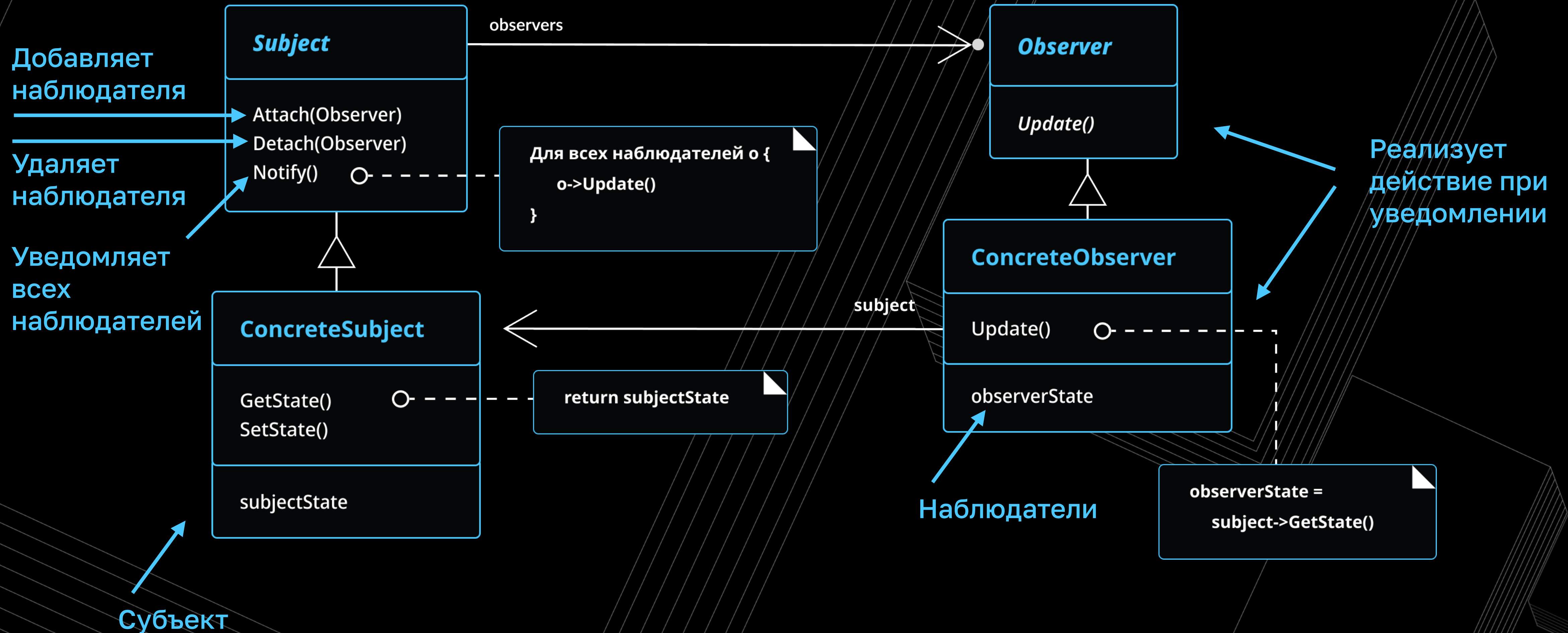
Advanced Message Queuing Protocol



Решаемая задача

- Поддерживает в согласованном состоянии взаимосвязанные объекты
- Уменьшает связанность объектов (Low Coupling)

Интерфейсы



Интерфейс
субъекта

```
interface ISubject
{
    public function attach(IObserver $observer);

    public function detach(IObserver $observer);

    public function notify();
}
```

Добавляет
наблюдателя

Удаляет
наблюдателя

Уведомляет всех
наблюдателей
о новом событии

Субъект

```
class ConcreteSubject implements ISubject
{
    private \SplObjectStorage $observers;

    public function __construct()
    {
        $this->observers = new \SplObjectStorage();
    }

    public function attach(IObserver $observer): self
    {
        $this->observers->attach($observer);
        return $this;
    }

    public function detach(IObserver $observer): self
    {
        $this->observers->detach($observer);
        return $this;
    }

    public function notify(): self
    {
        foreach ($this->observers as $observer) {
            $observer->update();
        }
        return $this;
    }
}
```

Добавляет
наблюдателя

Удаляет
наблюдателя

Уведомляет всех
наблюдателей
о новом событии

Интерфейс
наблюдателя

```
interface IObserver  
{  
    public function update();  
}
```

Обработка
события

Наблюдатель

```
class ConcreteObserver implements IObserver
{
    public function update()
    {
        echo 'Обрабатываем событие.' . PHP_EOL;
    }
}
```

Наблюдателей
может быть много

```
class AnotherConcreteObserver implements IObserver
{
    public function update()
    {
        echo 'Обрабатываем другое событие.' . PHP_EOL;
    }
}
```

```
function testObserver()  
{
```

```
    $observer = new ConcreteObserver();  
    $anotherObserver = new AnotherConcreteObserver();
```

```
    $concreteSubject = new ConcreteSubject();  
    $concreteSubject->attach($observer);  
    $concreteSubject->attach($anotherObserver);
```

```
    $concreteSubject->notify();
```

```
}
```

Создаём
наблюдателей

Добавляем
наблюдателей
в субъект

Уведомляем
наблюдателей

Плюсы

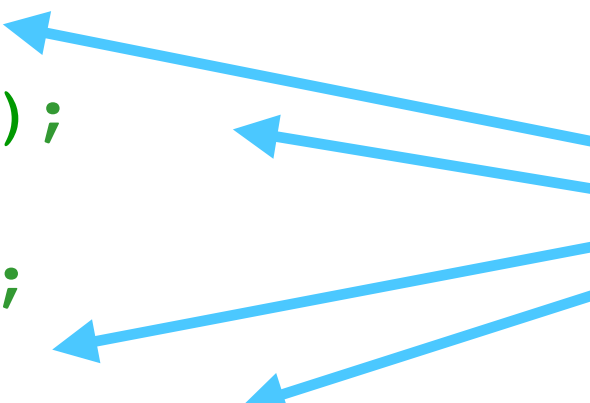
- Убирает жёсткую связанность между отправителем и получателем
- Легко добавить наблюдателей в runtime
- Всех отправителей и наблюдателей просто сочетать в любой комбинации

```
$observer = new ConcreteObserver();  
$anotherObserver = new AnotherConcreteObserver();
```

```
$concreteSubject1 = new ConcreteSubject1();  
$concreteSubject1->attach($observer);  
$concreteSubject1->attach($anotherObserver);
```

```
$concreteSubject2 = new ConcreteSubject2();  
$concreteSubject2->attach($observer);  
$concreteSubject2->attach($anotherObserver);
```

Всем субъектам
можно добавить
любого наблюдателя



Минусы



- Если подписчиков немного, выгоднее воспользоваться более простым паттерном (например, Посредник)
- Сложность дебага кода и понимания, какие были выполнены операции
- Много времени занимает разработка системы хранения подписчиков между пользовательскими запросами

Оповещает
слушателей
(наблюдателей)
при изменении
состояния системы

CHAIN OF RESPONSIBILITY



ЦЕПОЧКА ОБЯЗАННОСТЕЙ

Позволяет избежать привязки
отправителя запроса к его
получателю, давая шанс обработать
запрос несколькими объектам

Связывает объекты-получатели
в цепочку и передает запрос
вдоль этой цепочки,
пока его не обработают

Решаемая задача

- Разделяет объект, инициирующий операцию, от его обработчика
- Получатель запроса всегда анонимный
- Перемещает запрос по цепочке объектов, пока один из них не обработает его

Применимость

- Более одного объекта может обработать запрос
- Обработчик заранее неизвестен и должен быть найден автоматически
- Запрос отправляется одному из нескольких объектов, не указывая явно, какому именно
- Набор объектов, способных обработать запрос, должен задаваться динамически

Клиент
(инициирует запрос)

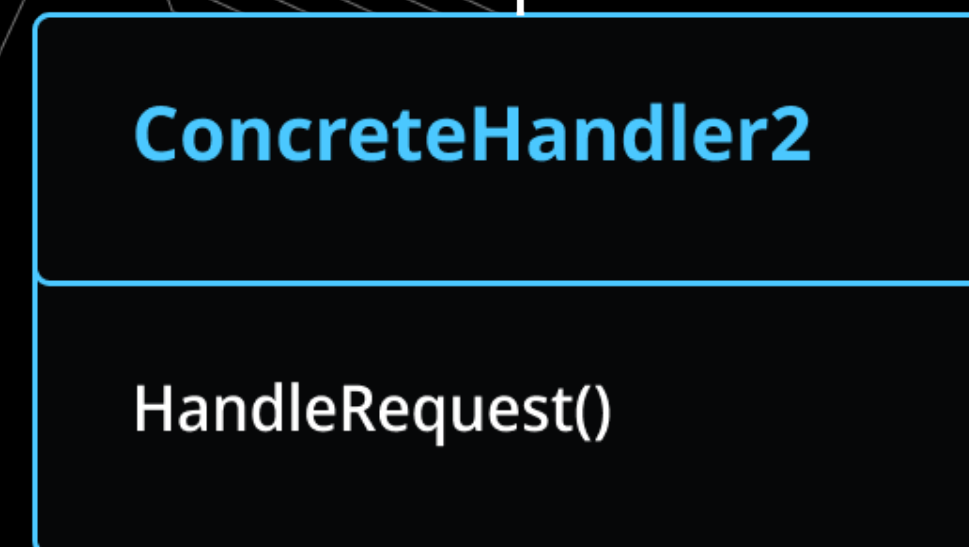
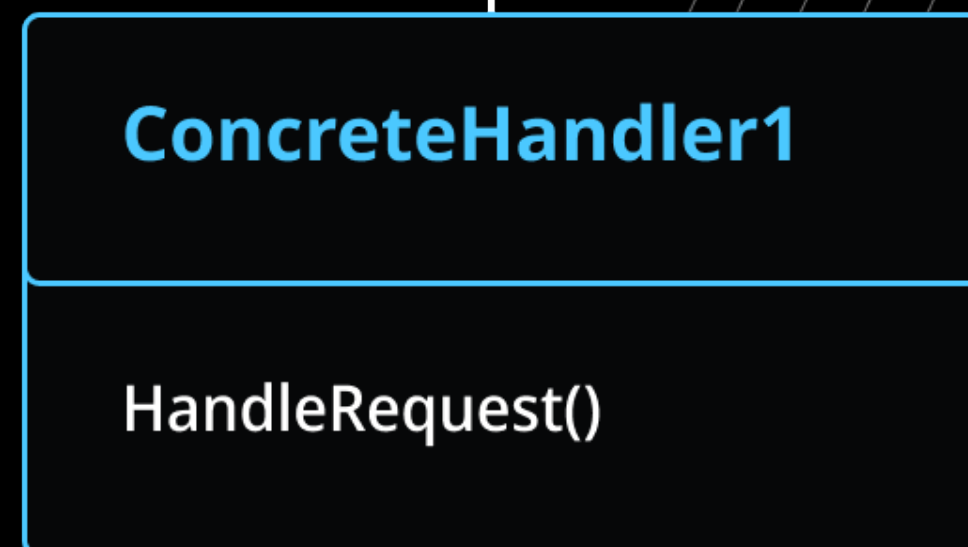


Обработчик
(интерфейс)



Рекурсивно
оборачиваем
все обработчики

Преемник



Конкретный обработчик

Обработчик

```
abstract class Handler
{
```

```
    private ?Handler $next = null;
```

Ссылка на
следующий обработчик

Формирует цепочку
обработчиков

```
    public function setNext(Handler $next): self
```

```
    {
```

```
        $this->next = $next;
```

```
        return $next;
```

Возвращается не текущий,
а добавленный обработчик

```
    }
```

```
    public function handleRequest(string $message): ?string
```

```
    {
```

```
        if ($this->next !== null) {
```

```
            return $this->next->request($message);
```

```
        }
```

```
        return null;
```

```
    }
```

```
}
```

Основной метод
для каждого
обработчика

Конкретные
обработчики

```
class ConcreteHandler1 extends Handler
{
    protected function handleRequest(string $message): ?string
    {
        if (strlen($message) < 6) {
            return 'Выполнился обработчик 1' . PHP_EOL;
        }

        return parent::handleRequest($message);
    }
}

class ConcreteHandler2 extends Handler
{
    protected function handleRequest(string $message): ?string
    {
        if (strlen($message) == 6) {
            return 'Выполнился обработчик 2' . PHP_EOL;
        }

        return parent::handleRequest($message);
    }
}

class ConcreteHandler3 extends Handler
{
    protected function handleRequest(string $message): ?string
    {
        if (strlen($message) > 6) {
            return 'Выполнился обработчик 3' . PHP_EOL;
        }

        return parent::handleRequest($message);
    }
}
```

Проверка на
ВОЗМОЖНОСТЬ
выполнения
обработчика

Идём по цепочке,
пока не найдётся первый
подходящий алгоритм

```
function testChainOfResponsibility()
{
    $chain1 = (new ConcreteHandler1());
    $chain2 = $chain1->setNext(new ConcreteHandler2());
    $chain3 = $chain2->setNext(new ConcreteHandler3());

    echo $chain1->handleRequest('apple');
    echo PHP_EOL;
    echo $chain1->handleRequest('banana');
    echo PHP_EOL;
    echo $chain1->handleRequest('watermelon');
}
```

Выстраиваем цепочку
из обработчиков

Результат:
Выполнился обработчик 1

Результат:
Выполнился обработчик 2

Результат:
Выполнился обработчик 3

Плюсы

- Неизвестно кто обрабатает запрос
- Отправитель и получатель ничего не знают друг о друге
- Просто добавить или убрать участников цепочки
- Уменьшает связанность объектов (low coupling)
- Реализует принцип персональной ответственности
- Реализует принцип открытости/закрытости

Минусы



- Нет гарантий, что запрос будет обработан
- Только один алгоритм может обработать событие

Отделяет
инициатора
от получателя
путём передачи
операции
по цепочке
объектов

Chain Of Responsibility

Decorator

- Расширяет исходный объект
- Не может прерывать выполнение декораторов

- Обработчики выполняют произвольные действия
- Может прерывать выполнение цепочки

Chain Of Responsibility

- Передаёт запрос по цепочке обработчикам

Mediator

- Убирает связь между объектами и производит общение через себя

Command

- Устанавливает связь между отправителем и получателем

Observer

- Передаёт запрос всем получателям, позволяя им динамически подписываться

avito.tech

Смотрите все серии на сайте
<https://avito.tech/patterns>

